



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Artificial Intelligence and Data Science

CSA1008 – Software Engineering

Assignment Title: CO2_AT2-Architecture Design

Case Study Title: Smart Waste Collection System

(SDG 11: Sustainable Cities and Communities)

Name: K. Dedeepya

Reg No: 192324252

Class & Section: 4th year B. Tech AIDS

Faculty Name: Sharmili K

Date of Submission: 01-08-26

1. System Requirement Analysis

1.1 System Objectives

The Smart Waste Collection System aims to improve municipal waste management by monitoring the fill level of garbage bins in real time, optimizing waste collection routes, reducing operational costs, preventing overflowing bins, and providing efficient services to citizens.

1.2 Functional Requirements

- User login and authentication
- Monitor smart bin fill levels using IoT sensors
- Display bin locations on a city map
- Generate optimized collection routes
- Assign routes to drivers
- Track collection vehicle locations
- Update collection status

1.3 Non-Functional Requirements

- High availability (24×7 operation)
- Fast response time
- Scalability for thousands of smart bins
- Secure authentication
- Reliable data transmission
- Easy maintenance

1.4 Quality Attributes

Quality Attribute	Importance
Scalability	Supports increasing number of bins and users
Maintainability	Easy software updates and modifications
Reliability	Accurate sensor data and uninterrupted service
Availability	Continuous system access
Performance	Fast route generation and monitoring
Security	Protect user accounts and operational data

2. Selection of Suitable Software Architecture

Selected Architecture: Layered Architecture with Client–Server Architecture (Hybrid Architecture)

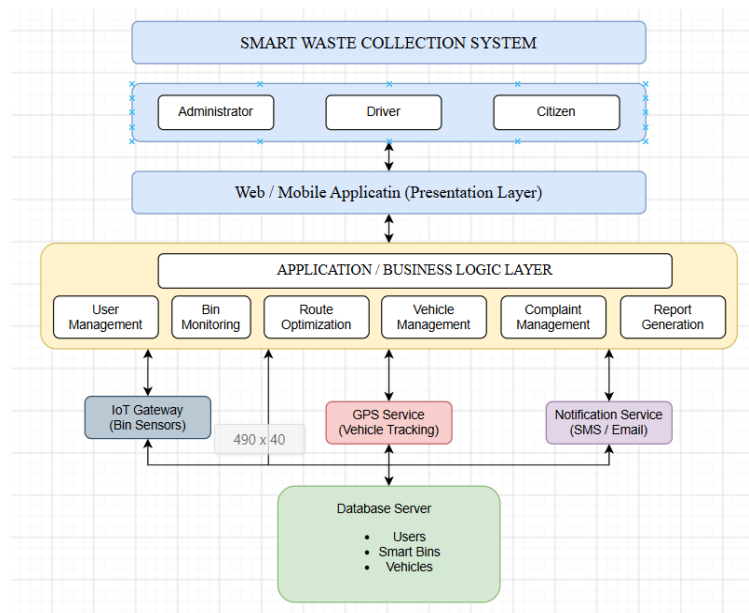
Justification

The Smart Waste Collection System contains multiple users such as administrators, drivers, citizens, and IoT devices.

A hybrid architecture is appropriate because:

- separates presentation, business logic, and database layers
- simplifies maintenance
- supports future expansion
- enables secure communication
- supports web and mobile applications
- easily integrates IoT sensors and GPS services

3. Software Architecture Diagram



4. Components and Their Interactions

Presentation Layer

Provides interfaces for administrators, drivers, and citizens using web and mobile applications.

Responsibilities

- Login
- Dashboard
- View maps
- Route information
- Complaint submission

- Reports

Business Logic Layer: Processes all application logic.

Responsibilities

- Authentication
- Route optimization
- Vehicle assignment
- Bin monitoring
- Complaint processing
- Report generation

IoT Gateway

Collects data from smart waste bin sensors and forwards it to the application server.

GPS Service

Tracks collection vehicle locations and assists in route optimization.

Notification Service

Sends alerts when:

- bins are full
- routes change
- complaints are resolved

Database

Stores:

- Users
- Smart bins
- Sensor readings
- Vehicles
- Drivers
- Routes
- Complaints
- Reports

Workflow

1. Smart bin sensors measure waste levels.

2. Sensor data is transmitted through the IoT gateway.
3. The server stores sensor data.
4. When a bin reaches the threshold, the system generates an alert.
5. The route optimization module calculates the best collection route.
6. Drivers receive optimized routes.
7. GPS updates vehicle locations.
8. Drivers update collection status after servicing bins.
9. Administrators monitor dashboards and generate reports.

5. Quality Attribute Analysis

Scalability

Supports thousands of smart bins, vehicles, and users without significant performance degradation.

Security

- User authentication
- Role-based access control
- HTTPS communication
- Encrypted passwords
- Secure APIs

Performance

- Real-time sensor updates
- Fast route calculation
- Efficient database queries

Reliability

- Backup databases
- Fault-tolerant communication
- Error handling for sensor failures

Maintainability

- Layered architecture enables independent updates to UI, business logic, and database.
- Modular components simplify debugging and testing.

Availability

- 24/7 monitoring

- Cloud deployment
- Automatic backups
- Redundant servers

6. Architectural Justification

The hybrid architecture combining Layered and Client–Server models is the most suitable because it separates user interfaces, application logic, and data storage into independent layers. This improves modularity, simplifies maintenance, and supports future expansion.

Architectural Decisions

- Layered architecture improves maintainability.
- Client–Server architecture supports centralized data management.
- IoT integration enables real-time monitoring.
- GPS services enhance route optimization.
- Notification services provide timely alerts.

Trade-offs

- Slightly higher implementation complexity due to multiple layers.
- Additional infrastructure required for IoT devices and cloud services.
- More network communication compared to a monolithic application.

Future Enhancements

- AI-based waste level prediction.
- Machine learning for dynamic route optimization.
- Integration with smart city platforms.
- Citizen mobile notifications.
- Recycling analytics dashboard.